CS6135 VLSI Physical Design Automation
Homework 2: Two-way Min-cut Partitioning

110062110 周芷瑄

How to compile and execute the program:

1. "cd /HW2_grading/student/110062110/HW2/src "
   (Enter the folder where makefile is)
2. "make"
   (Compile and generate the executable file into the folder bin)
3. "../bin/hw2 ../testcase/publicX.txt ../output/publicX.out"
   or just "make run X"
   (X is the number of the testcase, this execute the program with the given file as input/output, the output will be in the folder output)

Screenshot of the result:

```
+-----------------------------------------------+
|                                               |
|    This script is used for PDA HW2 grading.   |
|                                               |
+-----------------------------------------------+
host name: ic21
compiler version: g++ (GCC) 7.3.0

grading on 110062110:
 checking item          | status
------------------------|--------
 correct tar.gz         | yes
 correct file structure | yes
 have README            | yes
 have Makefile          | yes
 correct make clean     | yes
 correct make           | yes

  testcase |   cut size |    runtime | status
----------|------------|------------|--------
   public1 |       1043 |       0.14 | success
   public2 |        134 |       0.10 | success
   public3 |      31622 |       2.86 | success
   public4 |     167907 |       4.21 | success
   public5 |     154100 |      79.95 | success
   public6 |     151612 |     117.34 | success
+-----------------------------------------------+
|                                               |
|   Successfully write grades to HW2_grade.csv  |
|                                               |
+-----------------------------------------------+
```

Algorithm:

I implemented the FM algorithm taught in the class, ([A Linear-Time Heuristic for Improving Network Partitions])

Pseudo code and description of some functions:

- Initial partition:

There are two version of initial partition in my implementation.

```
bool initPartition_v2() {
    initialize the size of Die_A and Die_B to 0

    sort the nets by weight from heavy to light --> sorted_nets

    put the cells into a list by the order of appearance in sorted_nets --> cell_list
    put all cells into A first

    for(all cell in cell_list)
        keep moving the cell from DieA to DieB until meeting the constrain

    if the constrain is met, then return true
    else return false

}
```

我先把 nets 按照 weight 大小排好，再將每個 net 有連到的 cell 一個一個放進 cell_list，如果 cell 已經被放過了，就會被跳過。
將每個 cell 都先放到 A 裡面，再一個一個搬到 B 直到符合使用率的要求。
我一開始想說這樣的話，最初的 cut size 可以小一點，但發現有很多 case 用這個 function 是分不了的。

```
void initPartition() {
    initialize the size of Die_A and Die_B to 0
    sort the cells by size in B from small to big --> sorted_cells

    put all cells into A first

    for(all cell in sorted_cells)
        keep moving the cell from DieA to DieB until meeting the constrain
}
```

當剛剛提到的方式分不了時，會進行這個版本的 partition。首先會將 cell 按照再 Die_B 裡面的大小排好並全部放入 Die_A 中，再一個一個從 A 搬到 B 直到符合條件。這個方式再 public cases 都可以分。
也有另一個 function 是先把 cells 都放到 B 再一個一個搬到 A，如果上述的 partition 沒有過的話就會執行另一個。

其餘的實作基本上就是老師上課教的 FM 演算法，就一步一步跟著寫。

```
while (true) {
    int best_gain = pass();

    if (best_gain <= 0) {
        break;
    }

    for (auto& c : dataMgr->AllCells) {
        c->locked = false;
    }
    ++round;
}
```

pass 會回傳整個 FM 回合裡最大 gain 的 prefix sum，如果小於等於 0 的話表

示目前的 partition 沒辦法再更好了，就會跳出迴圈，結束計算。

```
while (true) {
    Cell* basecell = findBase();
    if (!basecell) break;

    // cumulative_gain = the gain improved by moving basecell
    cumulative_gain += basecell->gain;
    move_sequence.push_back(basecell);

    if (cumulative_gain >= max_gain) {
        max_gain = cumulative_gain;
        // move_sequence from index 0 to index best_move_index gives the best gain
        best_move_index = move_sequence.size() - 1;
    }
    updateCellGain(basecell);
}

// for all move_seq with index < best_move_index, roll back
for (int i = move_sequence.size() - 1; i > best_move_index; --i) {
    auto& record = move_sequence[i];
    if (record->in_dieA) {
        dataMgr->sizeA -= record->sizein_A;
        dataMgr->sizeB += record->sizein_B;
    }
    else{
        dataMgr->sizeA += record->sizein_A;
        dataMgr->sizeB -= record->sizein_B;
    }
    record->in_dieA = !record->in_dieA;
}
```

在每個回合，會重複找 gain 最大的 cell，如果他交換過去可以符合使用率的話就會讓他搬過去，然後計算 prefix sum 並且記錄最大值的 index。如果都找不到便會跳出迴圈。迴圈結束後會進行退回，把最大值出現的 index 之後的每顆 cell 都搬回去對面。這樣會是一個 pass。

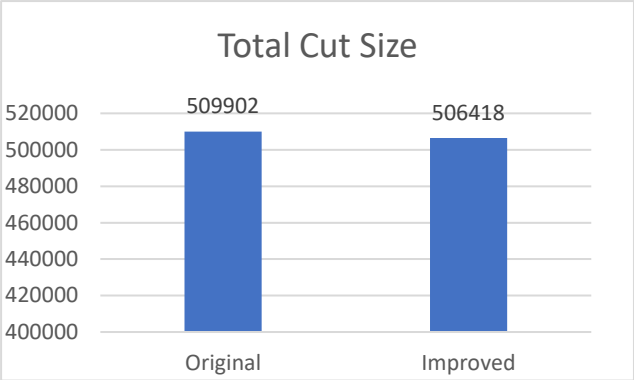
Techniques I tried

- **Initial partition**

testcase	cut size	runtime	status
public1	4024	0.08	success
public2	637	0.08	success
public3	31622	2.84	success
public4	167907	4.15	success
public5	154100	79.21	success
public6	151612	140.07	success

→

testcase	cut size	runtime	status
public1	1043	0.14	success
public2	134	0.11	success
public3	31622	2.88	success
public4	167907	4.23	success
public5	154100	79.92	success
public6	151612	140.12	success



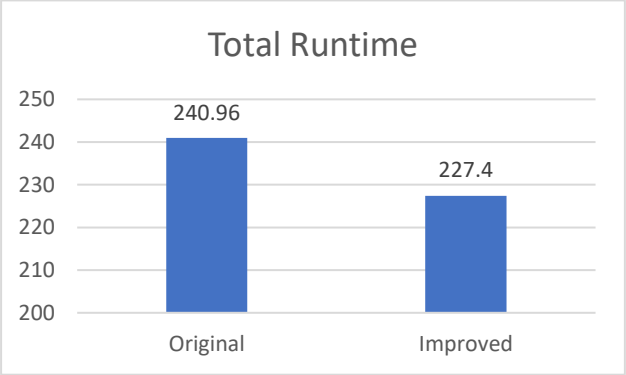
加總來看，使用 initPartition，initPartition_v2 的 cut size 比較小，但若細看實際數字，可以發現在 public testcase3 ~6，initPartition_v2 是不會有幫助的，原因是該 function 沒有辦法給予這些測資合法的初始擺放。

- **Unordered_map -> vector**

testcase	cut size	runtime	status
public1	1043	0.16	success
public2	134	0.12	success
public3	31622	2.96	success
public4	167907	4.42	success
public5	154100	83.96	success
public6	151612	149.34	success

→

testcase	cut size	runtime	status
public1	1043	0.14	success
public2	134	0.11	success
public3	31622	2.88	success
public4	167907	4.23	success
public5	154100	79.92	success
public6	151612	140.12	success



在 class BucketList 中，我先用 cell_info(unordered_map<Cell*, array<int, 2>>)記住每個 cell 在 list(vector<vector<Cell*>>)中的位子，其中包含 gain 值，也就是 list 的 entry、以及 idx，即在該 entry 中的哪個位子。E.g,

cell_info[cell][0] = the bucket index of the cell,

cell_info[cell][1] = index of the cell inside the bucket。

後來把 cell_info 改用 vector<array<int, 2>>實作，降低了一點點時間，但不非常顯著。經過查證，這兩個的 look up time complexity 都是 O(1)。

What I have learned

- 對 FM 演算法更熟悉
- 學會管理檔案
- 理解一些資料結構的運作方式及時間複雜度